

VeoNyx - Relatório para Portfólio

Resumo limpo do projeto para GitHub e apresentação profissional

Versão documental: v0.4

Data: Junho de 2026

Status: Projeto em desenvolvimento / QA local

Documento consolidado para organizar visão de produto, arquitetura, estado atual, roadmap, QA e diretrizes de desenvolvimento do VeoNyx.

1. Visão geral

VeoNyx é um projeto de aplicativo mobile-first de comunidades digitais criado para fãs de anime, RPG, games, fandoms, arte e grupos criativos. A proposta é oferecer comunidades com identidade própria, perfis locais personalizados, posts, wikis, chats, loja estética e ferramentas de moderação.

O projeto está em desenvolvimento local e ainda não representa uma versão de produção. O objetivo do portfólio é demonstrar arquitetura, evolução do produto, decisões técnicas e capacidade de desenvolvimento full-stack.

2. Problema que o projeto busca resolver

- Plataformas de comunidade como Amino e Project Z deixaram usuários sem uma alternativa moderna, estável e organizada.
- Apps genéricos como Discord e Reddit nem sempre valorizam identidade visual por comunidade.
- Comunidades de fandom, RPG e criação precisam de perfis locais, posts, wikis, chats e moderação contextual.
- Produtos sociais com público jovem precisam crescer com segurança, QA e regras claras.

3. Diferenciais do VeoNyx

Diferencial	Descrição
Perfil global + perfil local	O usuário tem uma identidade geral e pode ter uma identidade diferente em cada comunidade.
Tags comunitárias	Tags livres e tags de equipe aparecem no perfil local com limites e permissões.
Comunidade como sub-app	A navegação muda quando o usuário entra em uma comunidade.
Conteúdo estruturado	Posts, comentários, wikis, salvos e denúncias.
Moderação comunitária	Ações de staff, logs, denúncias, banimento e punições temporárias.
Seed QA	Dados de teste para validar comunidades com membros reais no ambiente local.

4. Stack técnica

Camada	Tecnologia
Mobile	Flutter / Dart
Backend	NestJS / TypeScript
Banco de dados	PostgreSQL
ORM	Prisma
Cache/tempo real	Redis
Storage local	MinIO
Testes e QA	flutter analyze, flutter test, build APK, seed QA, testes manuais no celular

5. Funcionalidades implementadas até agora

- Autenticação e sessão com JWT.
- Perfil global com avatar, banner, bio e wallet.
- Criação, edição, entrada e saída de comunidades.
- Perfil local por comunidade com avatar, banner, bio, cargo e tags.
- Posts, comentários, curtidas e salvos.
- Wikis com capa e detalhe.
- Chats públicos por comunidade.
- Notificações internas.
- Loja cosmética inicial.
- Tags livres e tags de equipe com regras de permissão.

- Moderação com denúncias, logs, warn, ban e punições temporárias.
- Seed QA para testar comunidades com membros de exemplo.

6. Arquitetura simplificada

Fluxo local de desenvolvimento:

Flutter App -> NestJS API -> Prisma -> PostgreSQL / Redis / MinIO

A separação mais importante do projeto é entre a identidade global do usuário e a identidade local dentro de cada comunidade.

7. Estado atual

Item	Status
Etapas 1 a 4	Implementadas.
Última rodada	Punições comunitárias, correção de tags de equipe e padronização de membros.
Validação automática	Backend validado; mobile precisa ser confirmado conforme última execução local.
Teste manual final	Pendente para a última rodada de correções.
Produção	Ainda não é versão de produção.

8. Roadmap resumido

- Validar a última rodada de correções no celular.
- Consolidar loja/cosméticos e economia visual.
- Implementar níveis, XP e check-in privado.
- Criar menu/central da comunidade.
- Evoluir suporte, tutoriais e categorias de conteúdo.
- Planejar RPG/personagens e lojas comunitárias.
- Executar macrofase futura de segurança, IA, bots, privacidade e Trust & Safety.

9. Aprendizados demonstrados no projeto

- Arquitetura full-stack com mobile, API, banco, cache e storage.
- Modelagem de domínio social com escopo global e local.
- Permissões comunitárias e moderação contextual.
- Uso de migrations e seeds com Prisma.
- QA incremental em aparelho real.
- Planejamento de produto inspirado em análise de concorrentes.
- Uso controlado de IA/Codex com prompts estruturados e validação por testes.

10. Observação para GitHub

Para publicação em portfólio, recomenda-se não expor senhas reais, tokens, .env, chaves, dados pessoais ou credenciais de teste. O repositório deve conter apenas código-fonte, documentação segura, exemplos de configuração e instruções de execução local.